



Distributed Movie Search

Course: Big Data Processing and Analytics

Students:

Melih Yelman, 21011702

Enes Kerim Kulpu, 22011627

1) Motivation, Goals and Implementation Environment

The goal of this project is to build a system for distributed keyword searching. It is designed for huge IMDb reviews that are too big for a single machine's RAM and disk to handle. The system splits the data and processes it in parallel. First, the search index is built as a batch process. After that, users can run queries and get results in milliseconds.

The working environment is a cluster set up using Docker, running linux/amd64 Hadoop images. For the cluster, we used the bde2020/hadoop 3.2.1 images, which include:

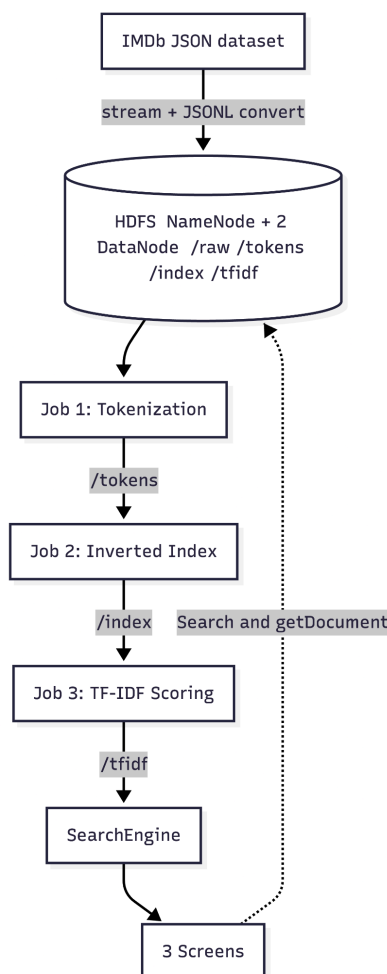
- 1 NameNode
- 2 DataNodes
- ResourceManager
- NodeManager
- JobHistory Service

Because the JVM inside these containers uses Java 8, the application was also built using Java 8 bytecode. Finally, the project uses the following tools:

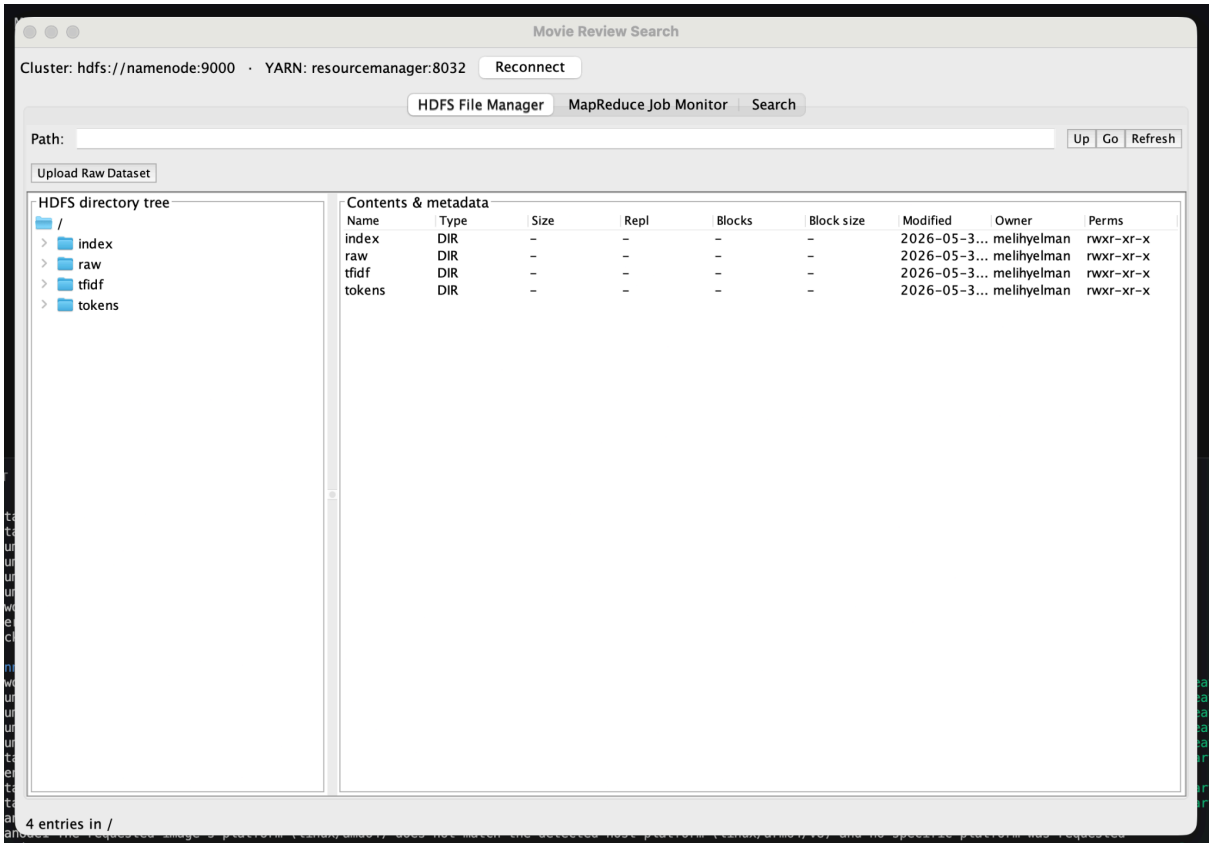
- Jackson for parsing JSON data.
- Java Swing for the user interface.
- Maven for packaging the application.

2) System Architecture and GUI Steps

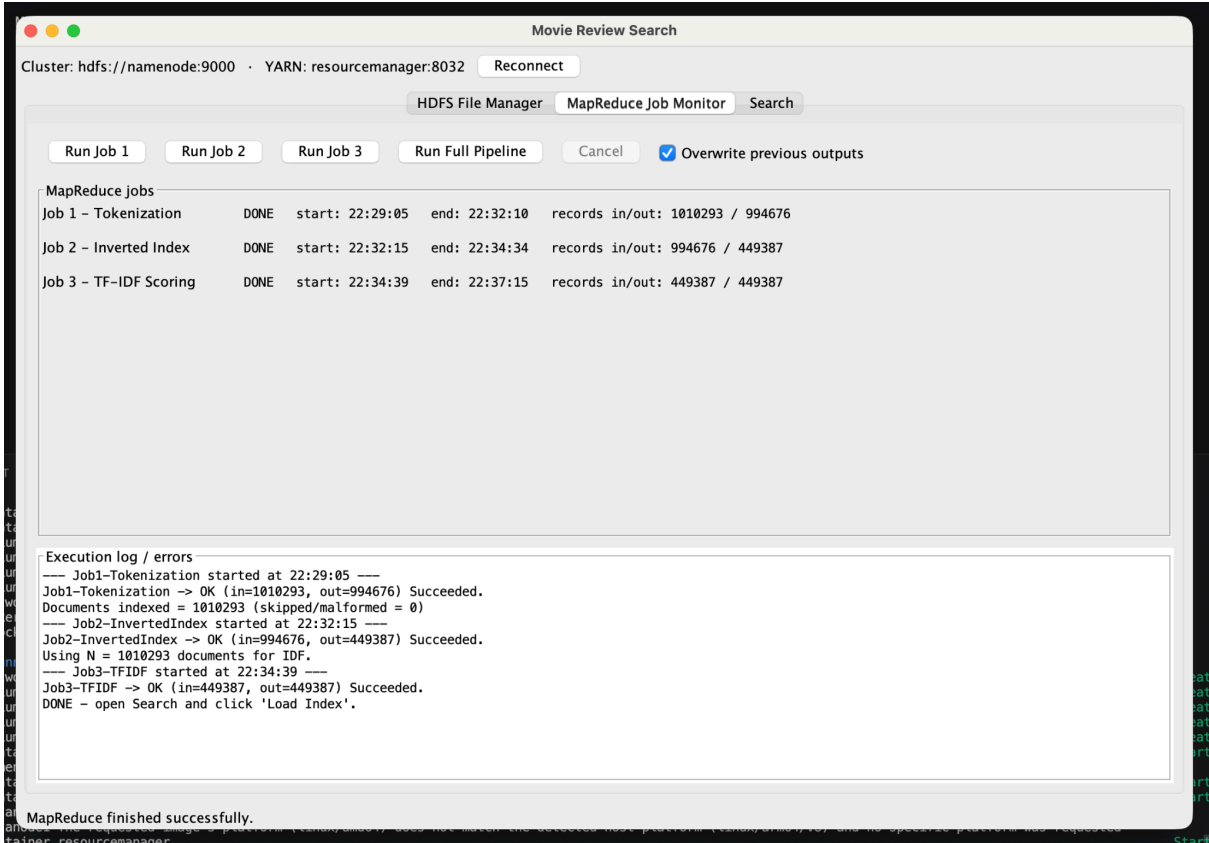
System Architecture like:



GUI Step 1, Click Upload Raw Dataset:



GUI Step 2, Run Full Pipeline:



GUI Step 3, Load index and run queries

Cluster: hdfs://namenode:9000 · YARN: resourcemanager:8032

Reconnect

HD FS File ManagerMapReduce Job MonitorSearch

Load IndexIndex loaded: 449387 terms in memory.

Query: gandalfSearch

Ranked results

Rank	Doc ID	Score	Movie
1	rw5579658	7.7644	The Lord of the Rings: The Fellowship of the Ring (2001)
2	rw6096705	7.3328	The Lord of the Rings: The Fellowship of the Ring (2001)
3	rw5736086	6.4248	The Lord of the Rings: The Two Towers (2002)
4	rw6405940	6.2566	The Hobbit: The Desolation of Smaug (2013)
5	rw6301847	5.8459	The Lord of the Rings: The Fellowship of the Ring (2001)
6	rw2922344	5.5856	The Hobbit: The Desolation of Smaug (2013)
7	rw2930626	5.5856	The Hobbit: The Desolation of Smaug (2013)
8	rw2925975	5.5856	The Hobbit: The Desolation of Smaug (2013)
9	rw2933435	5.5856	The Hobbit: The Desolation of Smaug (2013)
10	rw2924873	5.5856	The Hobbit: The Desolation of Smaug (2013)
11	rw2928997	5.5856	The Hobbit: The Desolation of Smaug (2013)

Original review (fetched from HDFS on click)

Doc ID : rw5579658

Movie : The Lord of the Rings: The Fellowship of the Ring (2001)

Rating : 10

Summary: the citizen kane of fantasy films

The lord of the rings: the fellowship of the ring is without doubt the best movie of 2001 , it has everything you need to make a great fantasy movie, the film's plot starts out with showing the second age in middle earth were the dwarves, men and elves get introduced to rings by Sauron who then forges his own ring to rule them all. which then results in a disaster, he uses it to take power so he can take over middle earth, he summons many servants including, orcs and his own wraiths. many of his servants attack lands and burn down villages capturing and killing people. nearly making the whole land under shadow , but then an alliance of men, and elves forms and they go fight off the orcs in middle earth , where they use archers and warriors, then Sauron himself comes and attacks the men, and then a man named Isildur who is the son of the king gets wounded with his father lying dead he takes his sword and defeats Sauron by cutting off his fingers, he is then told by the elf lord Elrond (played by Hugo weaving) to take the ring to the volcano mount doom to destroy it but Isildur easily falls under corruption of the ring and refuses to destroy it, the film is also starting with a narration of how the ring passed on to many others who also fall under its spell, it starts when Isildur got killed by remaining orcs, and there the ring falls under water and years later it is taken by a creature named gollum who calls it precious and he used it to disappoint his friends and relatives who later reject him out of the family , he then retreats to the mountain where he had lived over the thousand years , and many have thought of the ring as lost until 60 years later it has been found whether it was gollum or bilbo baggins played by ian holm who the ring is later carried by after when gollum lost the ring, then after the years pass bilbo celebrates his 111th birthday, where his nephew frodo baggins (played by Elijah wood) reads a book and then sees Gandalf (played by Ian mckellan) and meets him for a ride. and later drops Frodo off at his home. then the party of bilbo's birthday is celebrated,

50 results for "gandalf" – query time: 3 ms

Cluster: hdfs://namenode:9000 · YARN: resourcemanager:8032

Reconnect

HD FS File ManagerMapReduce Job MonitorSearch

Load IndexIndex loaded: 449387 terms in memory.

Query: robert de niroSearch

Ranked results

Rank	Doc ID	Score	Movie
1	rw2899060	12.4716	Heat (1995)
2	rw5323415	12.0415	烈火悍將 (1995)
3	rw3476835	11.9007	Dirty Grandpa (2016)
4	rw6130119	11.9007	Goodfellas (1990)
5	rw0977415	11.7283	The King of Comedy (1982)
6	rw1995085	11.2324	The Good Shepherd (2006)
7	rw3431168	11.1006	Raging Bull (1980)
8	rw2986834	10.9282	The King of Comedy (1982)
9	rw6157146	10.9282	The War with Grandpa (2020)
10	rw2899060	10.9282	The War with Grandpa (2020)

Original review (fetched from HDFS on click)

Doc ID : rw2899060

Movie : Heat (1995)

Rating : 10

Summary: An Excellent,Epic,Unforgettable And Powerful Crime Drama Masterpiece From Michael Mann. Michael Mann,Al Pacino And Robert De Niro At Their Best.

Heat is an excellent,epic,unforgettable and powerful Crime Drama Masterpiece that combines amazing direction,brilliant performances from Al Pacino,Robert De Niro and a great ensemble cast,thrilling Action,a fantastic script and a memorable score make Heat one of my favorite films,Michael Mann,Al Pacino and Robert De Niro at their best and a Masterpiece of cinema.Set in Los Angeles,California,Heat tells the story of Vincent Hanna(Al Pacino),an L.A.P.D. Homicide detective who's investigating a heist and the homicide a heist that was lead by Neil McCauley(Robert De Niro),an expert thief and robber that's in charge of a crew that helps Neil rob and steal and is really good. When Neil and his crew continue to rob and steal,Vincent is determined to take down Neil and his crew at all costs.Heat is a brilliant,unforgettable,epic Masterpiece that saw the paring of two amazing actors Al Pacino and Robert De Niro going head to head and although the two co-starred in The Godfather Part II(1974),Pacino and De Niro never appeared on screen together. From beginning to end Heat is an unforgettable Masterwork that gives viewers a film of cinematic power a movie that is a mixture of Crime Drama,Action movie and character study giving viewers a film that is unique and fresh making Heat one of the greatest Crime Dramas of all time but also the greatest Cops and Robbers film ever made. Heat is a movie that keeps you glued to the screen and is a film that is intense and thrilling,dark and gritty and at the same time vibrant and powerful with a tight grip that never lets go until the end. Heat is an epic film in the truest sense of the word not only in length(170 minutes)but the way it is shown with lots of characters and great storytelling and while the film is long you are Like most Michael Mann's films Heat is a movie that combines great style and substance the style is in the film looks thanks to Dante Spinotti's visually stunning photography using the colors blue and gray that gives the film a stylish look almost like a stylish but realistic Film Noir where the characters are surrounded by a darkness. The city of Los Angeles also plays a huge part Heat's style and look because the way that L.A. looks in this film because while there is

14 results for "robert de niro" – query time: 2 ms

3) MapReduce Jobs

Job	Mapper (in → out)	Reducer (in → out)	Output
1. Tokenization	(LongWritable, Text jsonLine) → (Text docId, Text word)	(Text docId, Iterable<Text> words) → (Text docId, Text "tok1 tok2 ...")	/tokens
2. Inverted Index	(Text docId, Text tokens) → (Text word, Text docId) (uses KeyValueTextInputFormat)	(Text word, Iterable<Text> docIds) → (Text word, Text "doc:tf,doc:tf,...")	/index
3. TF-IDF	(Text word, Text postings) → (Text word, Text "doc:tf") (for each posting)	(Text word, Iterable<Text> doc:tf) → (Text word, Text "doc:score,...") (sorted descending by score)	/tfidf

Additional Details:

- df (Document Frequency): This is the number of values that come into the Job 3 reducer.
- N (Total Documents): This is the STATS:DOC_COUNT counter from Job 1.
- Score Calculation: The TF-IDF score is calculated using the formula:

$$\text{tf-idf} = (1 + \log(\text{tf})) \times \log(N / \text{df})$$
- Text Cleaning: The TextAnalyzer is responsible for removing stopwords, converting text to lowercase, and removing non-alphanumeric characters. To ensure consistency, the exact same tokenizer is also applied to the user's search queries.

4) Performance Evaluation

Dataset Size	Documents	Job 1 Time	Job 2 Time	Job 3 Time	Total Pipeline Time	Term Count	Search Time
10 MB	8,672	~20.8 s	~21.5 s	~31.1 s	~2 min	48,113	1–3 ms
1 GB	~1 Million	~2 min	~2 min	~2 min	~6 min	~1M	10–50 ms

Performance Notes:

These measurements were taken on the same cluster using Docker Desktop (running with arm64 to amd64 emulation).

- Map vs. Reduce Phases: * Jobs 1 and 2: The "Map" phase does the heavy lifting because it handles parsing and tokenizing the text. The "Reduce" phase is much shorter since it only groups and combines the text.
 - Job 3: This is the opposite. The "Reduce" phase takes the most time because it has to calculate the IDF scores and sort everything.
- Number of Reducers (mapreduce.job.reduces): * If you use only 1 reducer, the total processing time takes longer.
 - If you increase it to 2 reducers, you gain some speed from parallel processing.

5) Discussion: Why is MapReduce not suitable for searching?

MapReduce has a built-in delay of several seconds for every job it runs. This is because it takes time to start up containers, load the JVM, and synchronize data during the "shuffle" phase. While this delay is fine for background "batch" processing, it makes MapReduce completely unsuited for interactive queries.

When a user types a word into a search box, they expect to see results in milliseconds.

Trying to run a brand new MapReduce job for every single search query would be too slow and is simply a bad architectural design.

The Solution:

Because of this limitation, our system separates the work into two steps:

- Building the Index: The heavy step (Jobs 1, 2, and 3) is done just once as a background MapReduce batch process to build the index.
- Searching: Once the index is built, user searches are processed using an in-memory HashMap. This allows the system to fetch results instantly, usually within microseconds to milliseconds.

6) Technical Challenges and Solutions

- Docker Communication: The biggest challenge was getting the host application to communicate properly with the Hadoop cluster running inside Docker. The NameNode returned DataNode addresses using container names, which the host system could not understand. Solution: We mapped the hostnames to the host machine and assigned a specific, published port to each DataNode.
- Classpath Issues: MapReduce jobs submitted from the host could not find the required Hadoop classes in the ApplicationMaster container. Solution: We fixed this by adding the absolute mapreduce.application.classpath and the HADOOP_MAPRED_HOME environment variables directly into the client configuration.
- Out of Memory Errors: Trying to load the 1 GB data index into memory caused an OutOfMemoryError. Solution: We limited the number of postings (documents) saved per term, added periodic hflush() calls when writing to HDFS, and started the application with a larger heap size. Together, these three changes reduced the memory usage for the full dataset down to just a few hundred megabytes.

7) Experience and Lessons Learned

- **The Power of Distributed Processing:** This project was a great opportunity to see the theoretical benefits of distributed computing in action. MapReduce allows each node to process its own data in parallel, meaning the system scales almost linearly as the data grows. Additionally, HDFS block-level replication provided both data safety (durability) and faster local reading.
- **Hadoop's Limitations:** The limitations of Hadoop also became very clear. Because there is a fixed setup time for every job, the delay is high even for small amounts of data. This perfectly shows why background batch indexing and fast interactive searching must be treated as two totally separate problems.
- **Configuration is Critical:** We learned that version and configuration matching is crucial in distributed systems. Since the configuration submitted by the client is pushed to all containers, a single mismatch can silently crash the entire pipeline. Most of the errors we encountered were not actual code bugs, but invisible configuration differences.
- **Final:** The Hadoop ecosystem is incredibly powerful when used for the right scenario like batch indexing big data. However, we saw in practice that as the scale of data gets larger, managing memory and I/O models becomes much more important than the algorithms themselves.